UP LT GRADE · COMPUTER SCIENCE MAINS

Computer Organization & Architecture Expected Numerical Questions with Full Solutions

One Worked Example From Every COA Question Type

1. Instruction Format Sizing · 2–4. Effective Address Calculation

5. Memory Address/Data Lines · 6. Memory Chip Count

7. Execution Time · 8. MIPS Calculation · 9. Average Memory Access Time (AMAT)

10. Amdahl's Law (Speedup) · 11. 2's Complement Overflow · 12. DMA Transfer Time · 13. PC Width

Practice set · step-by-step solutions · exam-style presentation

Table of Contents

TYPE 1 · INSTRUCTION FORMAT SIZING

Q1. Opcode Bits and Addressable Memory

Question

A CPU uses a 16-bit instruction format with 4 bits reserved for the opcode. (a) How many distinct operations can this CPU support? (b) How many bits remain for the address field, and what is the maximum directly addressable memory?

(a) Number of Distinct Operations

Opcode field = 4 bits → number of distinct operations = 2^4 = **16 operations**

(b) Remaining Bits and Max Addressable Memory

Total instruction length = 16 bits. Opcode uses 4 bits.
Remaining bits for address = 16 − 4 = **12 bits**
Maximum addressable memory = 2^{12} = **4096 memory locations**

Final Answer

(a) 16 distinct operations (b) 12 address bits → 4096 addressable locations

TYPE 2 · EFFECTIVE ADDRESS CALCULATION

Q2. Effective Address — Direct Addressing

Question

An instruction uses Direct Addressing with an address field value of 3000. Find the effective address and state how many memory accesses are needed to fetch the operand.

Solution

In Direct Addressing, the address field IS the effective address — no extra computation or pointer-chasing needed.
Effective Address (EA) = **3000**
Memory accesses needed to fetch operand = **1** (directly read location 3000)

Final Answer

EA = 3000, requiring only 1 memory access to retrieve the operand.

TYPE 3 · EFFECTIVE ADDRESS CALCULATION

Q3. Effective Address — Indirect Addressing

Question

An instruction's address field contains 500. Memory location 500 contains the value 2400. Using Indirect Addressing, find the effective address and the number of memory accesses required.

Step 1: Understand the Indirection

The address field (500) does NOT directly point to the operand. It points to a memory location that itself HOLDS the real address.

Step 2: Trace the Pointer

Read Memory Location 500 → contains value **2400**
This value (2400) is the actual Effective Address of the operand.

Step 3: Count Memory Accesses

Access 1: Read location 500 to obtain the pointer (2400)
Access 2: Read location 2400 to obtain the actual operand
Total = **2 memory accesses**

Final Answer

Effective Address = 2400; requires 2 memory accesses (one extra compared to Direct Addressing).

TYPE 4 · EFFECTIVE ADDRESS CALCULATION

Q4. Effective Address — Indexed/Base Addressing

Question

A Base register R contains the value 4000, and the instruction specifies an Offset/Index of 150. (a) Find the effective address under Indexed Addressing. (b) If the offset instead represents an array element NUMBER (not a byte offset) and each element is 4 bytes wide, recompute the effective address for element number 150.

(a) Direct Offset Case

Effective Address = Base Register + Offset
EA = 4000 + 150 = **4150**

(b) Array Element Case (Scaled Offset)

When the offset represents an ELEMENT NUMBER rather than a byte count, it must first be scaled by the element size:
Byte Offset = Element Number × Element Size = 150 × 4 = 600 bytes
Effective Address = Base + Byte Offset = 4000 + 600 = **4600**

Final Answer

(a) EA = 4150 (offset given directly in bytes) (b) EA = 4600 (offset scaled for 4-byte array elements)

TYPE 5 · MEMORY SIZING

Q5. Memory Address and Data Lines

Question

Find the number of address lines and data lines required for a memory of capacity 64K × 16.

Step 1: Address Lines (from number of locations)

64K = 65536 locations = 2^{16}
Address lines needed = **16**

Step 2: Data Lines (from word width)

Each location stores 16 bits → Data lines needed = **16**

Final Answer

16 address lines and 16 data lines are required for a 64K × 16 memory.

TYPE 6 · MEMORY CHIP COUNT

Q6. Building a Larger Memory from Smaller Chips

Question

Design a 64K × 16 memory system using 16K × 8 memory chips. How many chips are required in total, and how should they be arranged?

Step 1: Chips Needed to Extend Word Count (Address Range)

Required words = 64K, each chip provides 16K words.
Chips needed for word count = 64K ÷ 16K = **4** (arranged as 4 "rows", selected via extra address-decoding lines)

Step 2: Chips Needed to Extend Word Width

Required width = 16 bits, each chip provides 8 bits.
Chips needed for width = 16 ÷ 8 = **2** (arranged as 2 "columns", side-by-side to combine into a 16-bit word)

Step 3: Total Chip Count

Total chips = (chips for word count) × (chips for width) = 4 × 2 = **8 chips**
Arrangement: 4 rows × 2 columns

Final Answer

8 chips of 16K×8 are needed, arranged in a 4-row × 2-column grid to build a 64K×16 memory system.

TYPE 7 · CPU PERFORMANCE

Q7. Execution Time (IC × CPI × Clock Cycle Time)

Question

A program consists of 50,000 instructions. The average CPI (Cycles Per Instruction) is 2.5, and the CPU runs at a clock frequency of 1 GHz. Calculate the total execution time of the program.

Step 1: Find Clock Cycle Time

Clock Cycle Time = 1 / Clock Frequency = 1 / (1×10^9 Hz) = **1 nanosecond (ns)**

Step 2: Apply the Execution Time Formula

Formula

Execution Time = Instruction Count (IC) × CPI × Clock Cycle Time

Execution Time = 50,000 × 2.5 × 1 ns = **125,000 ns** = 125 µs = 0.125 ms

Final Answer

Total Execution Time = 125,000 ns (125 microseconds)

TYPE 8 · MIPS CALCULATION

Q8. MIPS (Millions of Instructions Per Second)

Question

Using the program from Q7 (50,000 instructions, executed in 125,000 ns), calculate the processor's performance in MIPS.

Step 1: Convert Execution Time to Seconds

125,000 ns = $125{,}000 \times 10^{-9}$ s = 1.25×10^{-4} s

Step 2: Apply the MIPS Formula

Formula

MIPS = Instruction Count / (Execution Time in seconds × 10^6)

MIPS = 50,000 / ($1.25 \times 10^{-4} \times 10^6$) = 50,000 / 125 = **400 MIPS**

Final Answer

The processor achieves **400 MIPS** for this program.

TYPE 9 · MEMORY HIERARCHY PERFORMANCE

Q9. Average Memory Access Time (AMAT)

Question

A system has a cache access time of 2 ns and a main memory access time of 20 ns. If the cache hit ratio is 90%, calculate the Average Memory Access Time (AMAT).

Step 1: Identify Given Values

Hit ratio (h) = 0.9 Miss ratio (1−h) = 0.1
Cache access time (Tc) = 2 ns Main memory access time (Tm) = 20 ns

Step 2: Apply the AMAT Formula

Formula

AMAT = (Hit Ratio × Cache Time) + (Miss Ratio × Main Memory Time)

AMAT = (0.9 × 2) + (0.1 × 20) = 1.8 + 2.0 = **3.8 ns**

Final Answer

Average Memory Access Time = 3.8 ns — much closer to cache speed (2 ns) than main memory speed (20 ns), showing why a high hit ratio matters so much.

TYPE 10 · MULTICORE PERFORMANCE

Q10. Amdahl's Law — Speedup from Parallelization

Question

A task takes 100 seconds on a single core. 80% of the task can be perfectly parallelized across 4 cores, while the remaining 20% must run sequentially. Find the overall speedup achieved.

Step 1: Identify Given Values

Parallelizable fraction (P) = 0.8 Sequential fraction (1−P) = 0.2 Number of cores (N) = 4

Step 2: Apply Amdahl's Law

Formula

Speedup = 1 / [(1−P) + P/N]

Speedup = 1 / [0.2 + 0.8/4] = 1 / [0.2 + 0.2] = 1 / 0.4 = **2.5×**

Final Answer

Overall Speedup = **2.5×** (even with 4 cores, the un-parallelizable 20% caps the maximum possible speedup well below 4×).

TYPE 11 · ALU ARITHMETIC

Q11. 2's Complement Overflow Detection

Question

Add the following 4-bit 2's complement numbers: 0111 (+7) and 0001 (+1). Check whether overflow occurs, and explain the rule used.

Step 1: Perform the Binary Addition

```
 0111  (+7)
+ 0001  (+1)
------------
 1000
```

Step 2: Interpret the Result

The result 1000 has MSB = 1, meaning it is interpreted as a NEGATIVE number (−8 in 4-bit 2's complement) — but two positive numbers (+7 and +1 = +8) can never produce a negative sum. This signals **overflow**.

Step 3: Apply the Overflow Rule

Overflow Detection Rule

Overflow occurs when the Carry INTO the sign bit is NOT EQUAL to the Carry OUT of the sign bit.
Alternative rule: Overflow occurs when two numbers of the SAME sign are added and the result has the OPPOSITE sign.

Final Answer

Result = 1000, and **overflow HAS occurred**, since two positive operands produced a result that appears negative. The true sum (+8) cannot be represented in 4-bit signed 2's complement (range: −8 to +7).

TYPE 12 · DMA / BUS TRANSFER

Q12. DMA Transfer Time

Question

A DMA controller needs to transfer a 1 MB file from disk to memory over a bus with a bandwidth of 100 MB/s. Calculate the total transfer time.

Step 1: Apply the Transfer Time Formula

Formula

Transfer Time = Data Size / Bus Bandwidth

Step 2: Substitute Values

Transfer Time = 1 MB / 100 MB/s = 0.01 s = **10 milliseconds (ms)**

Final Answer

Total DMA Transfer Time = 10 ms, during which the CPU is free to perform other tasks (only interrupted once, at completion).

TYPE 13 · REGISTER SIZING

Q13. Program Counter (PC) Width

Question

A computer has 1 MB of byte-addressable memory. How many bits are required for the Program Counter (PC) to address this entire memory?

Step 1: Express Memory Size as a Power of 2

1 MB = 1,048,576 bytes = 2^{20} bytes

Step 2: Determine PC Width

Since the PC must be able to hold the address of ANY byte in memory, and there are 2^{20} unique byte addresses, the PC must be **20 bits** wide.

Final Answer

The Program Counter must be **20 bits** wide to address all 1 MB of byte-addressable memory.

— End of Practice Set —